

EduRisk AI

Explainable Student Academic Risk Prediction & Intervention System

PROJECT REPORT

Student	Rahul Rahangdale
Programme	Integrated M.Tech in Artificial Intelligence
University	VIT Bhopal University
Domain	Artificial Intelligence / Machine Learning
Project Type	Machine Learning Classification Prototype

Academic project report based on the implemented EduRisk AI prototype.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. System Workflow
8. Dataset Description
9. Data Preprocessing
10. Machine Learning Models
11. Model Evaluation and Selection
12. Implementation
13. User Interfaces and Results
14. Testing
15. Challenges Faced
16. Learnings
17. Future Enhancements
18. Conclusion
19. References
Appendix A. Project Structure

1. Introduction

EduRisk AI is a machine learning prototype that classifies a student's academic risk as Low, Medium, or High using several academic and learning indicators. The project was developed as a complete AI/ML workflow rather than only a model-training exercise.

The system uses attendance, study hours, assignment score, internal marks, previous GPA, previous backlogs, sleep hours and class participation. Along with the prediction, it shows class probabilities, simple rule-based indicators and general academic support suggestions.

Dataset note: The dataset is synthetic and is included only for academic demonstration. The predictions should not be used as the sole basis for real academic decisions.

2. Problem Statement

Reviewing several student performance indicators manually can be time-consuming. EduRisk AI demonstrates how these indicators can be combined in a classification workflow to produce an initial academic risk category.

The project covers data loading, validation, preprocessing, model training, model comparison, prediction and a small intervention layer. Its purpose is to demonstrate AI/ML concepts in a usable software project.

3. Objectives

- Build an end-to-end student academic risk classification prototype.
- Use multiple academic and learning indicators as inputs.
- Compare three classification algorithms using standard evaluation metrics.
- Provide a command-line prediction interface.
- Provide a browser-based Streamlit dashboard.
- Show prediction probabilities, key indicators and general intervention suggestions.
- Keep the implementation modular, testable and documented.

4. Functional Requirements

Module	Function	Input	Output
Data loading	Load student records	CSV dataset	Loaded records
Validation	Check data and input ranges	Records / inputs	Validated data or error
Preprocessing	Prepare features for ML	Student indicators	Processed features
Model training	Train classifiers	Training data	Trained models
Model comparison	Compare candidate models	Labels + predictions	Accuracy/precision/recall/F1
Prediction	Classify an individual student	8 indicators	Low / Medium / High
Risk indicators	Find simple rule-based signals	Student indicators	Key indicators
Intervention	Suggest general support actions	Risk/indicators	Suggested actions
User interface	Accept and display results	CLI / browser	Readable prediction

5. Non-Functional Requirements

- Usability: results should be easy to understand for a student or faculty mentor.
- Performance: an individual prediction should be returned quickly after input.
- Reliability: invalid values should be checked instead of silently accepted.
- Maintainability: data, model, prediction and intervention logic are separated into modules.
- Resource efficiency: the prototype uses lightweight tabular ML models.
- Error handling: the input flow validates expected ranges and reports problems.

6. System Architecture

The system follows a modular pipeline. Data is loaded and validated, then prepared by preprocessing. Three classifiers are trained and compared. The selected model is saved and reused by both prediction interfaces.

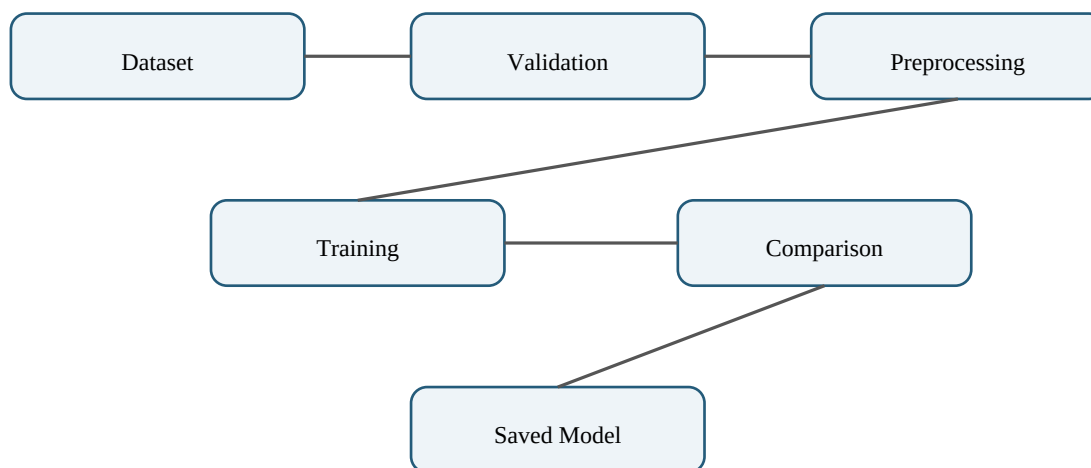


Figure 4: High-level system architecture

7. System Workflow

- Enter the student's academic and learning indicators.
- Validate the values and apply the same preprocessing used during training.
- Run the saved ML pipeline and obtain the predicted class and probabilities.
- Generate rule-based key indicators.
- Display general intervention suggestions.
- During training, compare candidate models and save the selected model.

Input → Validation & Preprocessing → ML Prediction → Risk + Probabilities → Indicators →
Intervention Suggestions

8. Dataset Description

The dataset contains 600 synthetic student records. It is used to demonstrate the software and machine-learning workflow and does not represent a real institutional student population.

Feature	Description
student_id	Record identifier
attendance_pct	Attendance percentage
study_hours_per_day	Average study hours per day
assignment_score_pct	Assignment performance percentage
internal_marks_pct	Internal assessment percentage
previous_gpa	Previous GPA on a 0–10 scale
previous_backlogs	Number of previous backlogs
sleep_hours	Average sleep hours per day
class_participation_pct	Class participation percentage
risk_level	Target: Low, Medium or High

9. Data Preprocessing

The preprocessing stage is shared by the candidate models. Numerical missing values are handled with imputation and numerical features are standardized. The preprocessing steps and classifier are kept together in a pipeline so the later prediction uses the same transformations as training.

- Separate the target column from the input features.
- Prepare the numerical feature matrix.
- Apply imputation where required.
- Standardize numerical features using StandardScaler.
- Pass the prepared features to the classifier.

10. Machine Learning Models

Logistic Regression

Used as a straightforward classification baseline for the tabular feature set.

Random Forest

Uses an ensemble of decision trees and provides a non-linear alternative to the baseline.

Gradient Boosting

Builds an ensemble sequentially and provides another tree-based comparison.

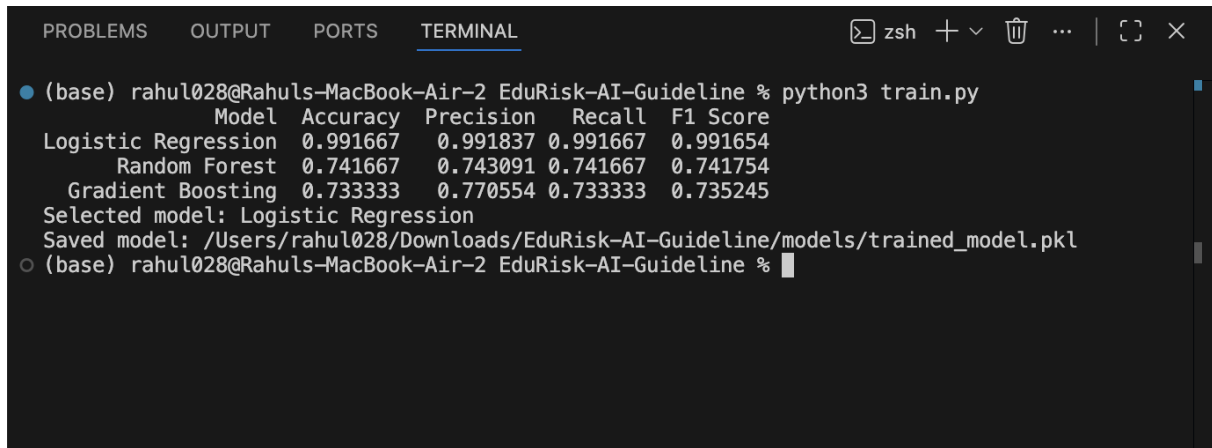
All three models use the same preprocessing pipeline so that the comparison is consistent.

11. Model Evaluation and Selection

The project compares accuracy, precision, recall and F1 score. The training script uses weighted F1 for model selection and saves the resulting prediction pipeline.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.991667	0.991837	0.991667	0.991654
Random Forest	0.741667	0.743091	0.741667	0.741754
Gradient Boosting	0.733333	0.770554	0.733333	0.735245

In the verified project run, Logistic Regression was selected by the implemented weighted-F1 comparison. Because the dataset is synthetic, these metrics should be interpreted only as results from this demonstration dataset.



```
(base) rahul028@Rahuls-MacBook-Air-2 EduRisk-AI-Guideline % python3 train.py
      Model Accuracy Precision Recall F1 Score
Logistic Regression 0.991667 0.991837 0.991667 0.991654
Random Forest      0.741667 0.743091 0.741667 0.741754
Gradient Boosting  0.733333 0.770554 0.733333 0.735245
Selected model: Logistic Regression
Saved model: /Users/rahul028/Downloads/EduRisk-AI-Guideline/models/trained_model.pkl
(base) rahul028@Rahuls-MacBook-Air-2 EduRisk-AI-Guideline %
```

Figure 6: Model training and evaluation output

12. Implementation

The project is split into separate files so that data handling, preprocessing, training, evaluation, prediction and intervention logic can be maintained independently.

File / Module	Purpose
train.py	Training, model comparison and saving
predict_cli.py	Terminal prediction interface
app.py	Streamlit browser interface
src/data_loader.py	Dataset loading
src/data_validation.py	Data validation
src/preprocessing.py	Preprocessing pipeline
src/model_training.py	Candidate model creation and training
src/model_comparison.py	Metric comparison
src/evaluation.py	Evaluation metrics
src/predictor.py	Saved-model prediction
src/explainability.py	Rule-based indicators
src/intervention.py	General intervention suggestions

13. User Interfaces and Results

13.1 Command-Line Prediction

The CLI accepts the eight indicators and prints the predicted risk, class probabilities, key indicators and suggested intervention.

```

EduRisk AI
Student Academic Risk Prediction System
=====
Enter the student's academic and learning indicators.

Attendance (%) [0-100]: 87
Study hours/day [0-12]: 2
Assignment score (%) [0-100]: 43
Internal marks (%) [0-100]: 34
Previous GPA [0-10]: 6
Previous backlogs [0-10]: 3
Sleep hours/day [0-12]: 3
Class participation (%) [0-100]: 20

-----
Predicted Academic Risk: HIGH
-----
Prediction Probabilities:
High : 100.00%
Low : 0.00%
Medium : 0.00%

Key Indicators:
- Assignment performance needs improvement.
- Internal assessment performance is relatively low.
- Previous GPA is below 7.
- Previous backlog(s) increase academic risk.
- Class participation is relatively low.

Suggested Intervention:
- Complete assignments earlier and review feedback.
- Revise internal-assessment topics and practice questions.
- Create a weekly plan for pending subjects.
- Discuss an academic support plan with a faculty mentor.

Note: This prototype uses a synthetic demonstration dataset.
Its prediction should not be used as the sole basis for real academic decisions.
(base) rahul028@Rahuls-MacBook-Air-2 EduRisk-AI-Guideline %

```

Figure 7: EduRisk AI command-line prediction output

In the demonstrated run, the entered values produced a High risk classification. The output also showed the probability distribution and the rule-based indicators that were triggered.

13.2 Streamlit Dashboard

The Streamlit dashboard provides the same prediction workflow in a browser. It uses input controls and presents the result visually with probabilities, key indicators and a suggested intervention.



Figure 8: EduRisk AI Streamlit dashboard

14. Testing

Test	Purpose	Result
test_data.py	Checks dataset/data behaviour	Passed
test_model.py	Checks model behaviour	Passed
test_prediction.py	Checks prediction workflow	Passed

The final local verification completed successfully with 3 passed tests. The training script also completed successfully and saved the trained model.

15. Challenges Faced

- Keeping preprocessing consistent between training and prediction.
- Organizing the code into reusable modules.
- Connecting both CLI and Streamlit interfaces to the same prediction logic.
- Making the output more useful than a single risk label.
- Clearly documenting the limitations of a synthetic dataset.

16. Learnings

The project gave practical experience with the complete ML workflow, from data preparation to a usable prediction interface. It also showed why software structure and testing matter even in a small machine-learning project.

- A preprocessing/model pipeline keeps training and prediction consistent.
- Comparing multiple models gives a clearer basis for model selection.
- Accuracy, precision, recall and F1 describe different parts of model performance.
- A simple UI makes an ML prototype easier to demonstrate.
- Small automated tests help catch basic issues before submission.

17. Future Enhancements

- Replace the synthetic dataset with a properly collected and ethically governed real-world dataset.
- Add stronger cross-validation and hyperparameter tuning.
- Add model-level explainability methods.
- Add secure prediction history if the project is extended into a real application.
- Add authentication and role-based access.
- Monitor data quality and model drift.
- Evaluate fairness and possible bias before any real-world use.

18. Conclusion

EduRisk AI demonstrates a complete machine-learning classification workflow for initial academic risk prediction. It loads and validates student indicators, preprocesses the data, compares three classifiers, saves the selected model and provides predictions through both a command-line interface and a Streamlit dashboard.

The project also adds rule-based indicators and general intervention suggestions to make the result easier to understand. The synthetic dataset is an important limitation, so the current system should be treated as an academic prototype rather than a real decision-making tool.

19. References

- Python documentation — programming language and standard library.
- scikit-learn documentation — preprocessing, classification and evaluation.
- Streamlit documentation — Python framework used for the dashboard.
- VITyarthi Build Your Own Project instruction document supplied for the course.

Appendix A. Project Structure

The main repository is organized into application files, data, models, source modules, tests and documentation.

```
EduRisk-AI/  
  ■■■ app.py  
  ■■■ predict_cli.py  
  ■■■ train.py  
  ■■■ requirements.txt  
  ■■■ README.md  
  ■■■ statement.md  
  ■■■ data/  
  ■   ■■■ student_performance.csv  
  ■■■ models/  
  ■   ■■■ trained_model.pkl  
  ■■■ src/  
  ■   ■■■ data_loader.py  
  ■   ■■■ data_validation.py  
  ■   ■■■ preprocessing.py  
  ■   ■■■ model_training.py  
  ■   ■■■ model_comparison.py  
  ■   ■■■ evaluation.py  
  ■   ■■■ predictor.py  
  ■   ■■■ explainability.py  
  ■   ■■■ intervention.py  
  ■■■ tests/  
  ■■■ docs/
```